BIT 326 - Distributed Systems
Chapter 1: Introduction

1.1 Definitions.

1) Distributed System
   - *A distributed system is a collection of independent computers that appears to its users as a single coherent system.*
   - *A collection of independent computers, or nodes, that work together and communicate over a network to achieve a common goal. From an external perspective, they appear to the end-user as a single, unified system.*
   - *A system in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages.*
   - *A collection of autonomous hosts that that are connected through a computer network. Each host executes components and operates a distribution middleware, which enables the components to coordinate their activities in such a way that users perceive the system as a single, integrated computing facility.*
   - *An application that executes a collection of protocols to coordinate the actions of multiple processes on a communication network, such that all components cooperate together to perform a single or small set of related tasks.*

   Also known as distributed computing and distributed databases, a distributed system is a collection of independent components located on different machines that share messages with each other in order to achieve common goals
   Distributed system will appear as if it is one interface or computer to the end-user.
2) Distributed Processing
   - *Distributed processing is a computing technique that divides tasks and disperses data across two or more interconnected, physically separate, or networked computers, allowing them to work together as a single system.*
     This approach increases computational power, enhances reliability, and enables handling massive datasets by parallelizing workloads
3) Distributed Operating System
   - *A distributed operating system (DOS) manages a collection of independent, networked, and physically separate computers, presenting them to users as a single, cohesive system.*
     It enables efficient resource sharing, improved performance, and high fault tolerance by allowing multiple CPUs to work together, even if one node fails.

1.2. Goals

The main goals of distributed systems are to provide

1) **Scalability:** The ability to handle an increasing workload by adding more resources (like servers) to the system without compromising performance.
2) **Reliability/Fault Tolerance:** Ensuring the system remains available and functional even if individual computers or components fail, by distributing work and data across multiple machines.
3) **Performance:** Executing tasks faster by processing them in parallel across multiple computers, leading to improved throughput and response times.

4) **Transparency:** Hiding the fact that resources and processes are physically distributed, so users perceive the system as a single, coherent whole.
5) **Openness:** Allowing the system to be extended and components to be added or replaced without disrupting the entire system, often by adhering to standard interfaces.
6) **Resource Sharing:** Making it easy and cost-effective for users and applications to access and share resources like hardware, software, and data that are located in different places.
7) **Concurrency:** Allowing multiple clients and processes to access resources simultaneously without interfering with one another.

1.3. Key Characteristics

Over the past years, distributed systems have gained substantial importance. The reasons of their growing importance are manifold:

1) **Resource Sharing:** Resource sharing means that the existing *resources* in a distributed system can be accessed or remotely accessed across multiple computers in the system. Computers in distributed systems shares resources like *hardware* (disks and printers), *software* (files, windows and data objects) and *data*. Resources are managed by a software module known as a resource manager. Every resource has its own management policies and methods.
2) **Geographically distributed environment**: First, in many situations, the computing environment itself is *geographically distributed*.
3) **Independence / collection of autonomous computing elements:** Nodes (computers) in the system operate independently and can fail independently of each other.
4) **Single coherent system**: A distributed system should appear as a single coherent system. In some cases, researchers have even gone so far as to say that there should be a single-system view, meaning that an end user should not even notice that processes, data, and control are dispersed across a computer network.
5) **Heterogeneity:** In distributed systems components can have variety and differences in Networks, Computer hardware, Operating systems, Programming languages and implementations by different developers.
6) **Openness:** Openness is concerned with extensions and improvements of distributed systems. The distributed system must be open in terms of *Hardware* and *Softwares*
7) **Concurrency:** The nodes operate concurrently, handling multiple requests and tasks simultaneously. Multiple tasks can be executed simultaneously across different nodes, which is crucial for handling high volumes of traffic.
8) **No global clock:** Nodes do not share a single, synchronized clock, which creates challenges for coordination due to network latency.
9) **Message passing:** Communication and coordination between nodes are achieved by passing messages over a network.
10) **Scalability:** Scalability is mainly concerned about how the distributed system handles the *growth* as the number of users for the system increases. A distributed system can be easily expanded by adding more nodes to handle increased demand, a process known as horizontal scaling.
11) **Fault Tolerance:** The system is designed to continue operating even if one or more individual nodes fail. Data can be replicated across multiple nodes, so if one goes down, a copy is still

available. A distributed system provides an excellent opportunity for incorporating *fault tolerance* and *graceful degradation*.

12) **Transparency:** The complexity of the underlying architecture is hidden from the user, who perceives it as a single, cohesive entity. Distributed systems should be perceived by users and application programmers as a whole rather than as a collection of cooperating components.
13) ***Speed up***: High processing speed. There is the need for *speeding up* computation, as the speed of computation in traditional uniprocessors is fast approaching the physical limit.

1.4. Evolution of Distributed Computing Systems – Enabling Technologies

Evolution of distributed computing systems has been rapid, and it has transformed way we use technology in our daily lives. From simple file sharing systems to advanced cloud computing, distributed computing systems have come a long way.

1) Early Days of Distributed Computing: The early days of distributed computing can be traced back to 1960s, where researchers began experimenting with idea of sharing resources across multiple computers. The concept gained popularity in 1970s, and early examples of distributed computing systems included file-sharing networks and email systems. In 1980s, focus shifted towards client-server architectures, where one computer acted as a server and provided services to multiple clients.
2) Peer-to-Peer Computing: The rise of peer-to-peer computing in late 1990s marked a significant shift in evolution of distributed computing systems. Peer-to-peer computing refers to a system where computers on a network can share resources with one another without need for a central server.
3) Grid Computing: Grid computing is another significant milestone in evolution of distributed computing systems. Grid computing refers to a system where a network of computers can work together to solve a complex problem. Grid computing is often used in scientific research, where large amounts of data need to be analyzed.
4) Cloud Computing: Cloud computing is perhaps most significant development in evolution of distributed computing systems. Cloud computing refers to a system where users can access resources, such as servers, storage, and applications, over internet. Cloud computing has transformed way businesses operate, allowing them to scale quickly and reduce costs.
5) Internet of Things: The Internet of Things (IoT) is a recent development in evolution of distributed computing systems. IoT refers to a network of interconnected devices, such as sensors and appliances, that can communicate with one another and share data. IoT relies on distributed computing systems to process vast amounts of data generated by these devices.
6) Edge Computing: Edge computing is a new trend in distributed computing systems that involves processing data closer to source rather than sending it to a central location. Edge computing is often used in applications where low latency is critical, such as self-driving cars and industrial automation. By processing data at edge of network, edge computing can reduce latency and improve performance.

1.5. Components of a Distributed System

A distributed system consists of multiple components located on different networked computers that communicate and coordinate their actions by passing messages. Here are the key components of a distributed system:

Hardware Components/Platform

1) **Nodes:** Individual computers or devices that participate in the distributed system.
   - Heterogeneous Hardware / General-Purpose Servers: The backbone for cloud computing, offering scalable compute and storage.
   - Edge Devices / Moble Devices & Embedded Systems: Smartphones, tablets, and IoT sensors forming large-scale, heterogeneous networks (Wireless Sensor Networks - WSNs).
   - Specialized Hardware: GPUs for parallel processing, large storage arrays for big data.
   - Cloud Infrastructure: Virtualized resources (VMs, containers) on large-scale data centers, forming the backbone for many distributed services (AWS, Azure, GCP).
   - Multiprocessor & Cluster**:** Systems with multiple CPUs sharing memory (bus/switch) or having private memory, forming the core of many data centers.
2) **Network:** The communication infrastructure that connects the nodes.
   - LAN, WAN, Internet, VPN.
3) **Data Stores:** Systems used for storing, retrieving, and managing data.
   - Databases (SQL, NoSQL), distributed file systems, object storage.

Software Components/ Platform

4) **Middleware:** Software layer that sits between the operating system and applications on each node, providing common services and enabling communication.
   - Message brokers, RPC frameworks, APIs, ORBs (Object Request Brokers).
5) **Processes:** Independent units of execution that can run on different nodes.
   - Microservices, worker processes, daemon processes.
6) **Services:** Functional units that provide specific capabilities or perform specific tasks, often accessible over the network.
   - Authentication service, database service, caching service.
7) **Communication Protocols:** Rules and standards that define how data is exchanged between nodes.
   - HTTP/HTTPS, TCP/IP, gRPC, AMQP, MQTT.

Tools & Technology

8) **Consistency Mechanisms:** Techniques used to ensure data consistency across the distributed system.
   - Two-phase commit, Paxos, Raft, eventual consistency.
9) **Load Balancers:** Components that distribute incoming network traffic across multiple servers to ensure no single server becomes overwhelmed.
   - Nginx, HAProxy, AWS Elastic Load Balancing.
10) **Coordination Services:** Services that help manage the state and configuration of distributed components.
   - ZooKeeper, etcd, Consul.
11) **Monitoring and Logging Systems:** Tools and services for tracking performance, availability, and logs of the distributed system.
   - Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana), Splunk.
12) **Security Components:** Mechanisms to protect the system from unauthorized access and ensure data integrity and privacy.
   - Authentication and authorization services, encryption, firewalls, VPNs.
13) **Scalability Components:** Features and tools that enable the system to handle increasing loads by adding more resources.

- Auto-scaling groups, container orchestration systems like Kubernetes, serverless functions.

14) **Fault Tolerance and Recovery Mechanisms:** Strategies and components that ensure the system continues to operate in the event of failures.
    - Replication, failover, backup and restore, redundancy.
15) **Scheduling and Orchestration Systems:** Tools that manage the deployment, scaling, and operation of containers and microservices.
    - Kubernetes, Docker Swarm, Apache Mesos.
16) **Distributed Ledger Technology:** Systems for achieving consensus and maintaining a shared, immutable record of transactions.
    - Blockchain, distributed databases.

1.6. Distributed Computing vs Parallel Computing

Distributed computing and parallel computing are often confused and might seem interchangeable. They both involve multiple processes working towards a single goal, but they are distinct approaches designed for different scenarios. Distributed computing focuses on using a network of separate machines, often geographically dispersed, to solve problems collaboratively. Each machine operates as an independent node with its own memory and processing power. Communication between nodes happens over a network, and tasks are divided so they can run on different machines.

Parallel computing, on the other hand, typically takes place within a single system. It uses multiple processors or cores within one computer to execute tasks simultaneously. These processors share the same memory and often work closely together to split up computations.

Both methods are incredibly useful but are suited to different tasks. Parallel computing is ideal for tasks that need high-speed computation with minimal communication overhead, like running simulations on a supercomputer. Distributed computing is better suited when scaling out, like when processing data in cloud-based systems or across global networks.

Aspect	**Centralized / Parallel**	**Distributed**
Scalability	Vertical scaling only (bigger Hardware)	Horizontal Scaling (more machines)
Failure Impact	Single point of failure brings down everything	Isolated failures, graceful degradation
Consistency	Strong consistency guaranteed	Eventual consistency, potential data lag
Development Complexity	Simple debugging and testing	Complex debugging across services
Deployment	Single deployment process	Independent service deployments
Network Dependency	No network between components	Network failures affect functionality
Data Management	Centralized Database with ACID guarantees.	Distributed data with BASE properties.

	(Atomicity, Consistency, Isolation & Durability)	**(Basically Available, Soft State, Eventually Consistent)**
Team Structure	Cross-functional teams share codebase	Independent teams own services
Monitoring	Simple application monitoring	Distributed tracing and correlation
Cost	Lower initial complexity costs	Higher operational and tooling costs

1.7. Challenges in Distributed Computing Systems

Despite benefits of distributed computing systems, there are also several challenges that need to be addressed. These challenges include

1) Security: Security is a significant concern in distributed computing systems, as data is transmitted and stored across multiple nodes. Cyber attacks such as denial of service (DoS) attacks, data breaches, and ransomware attacks are a constant threat.
2) Scalability: Distributed computing systems must be designed to scale to handle large workloads. However, scaling a distributed system can be complex and require significant engineering efforts.
3) Fault Tolerance: Fault tolerance is critical in distributed computing systems, as failures can occur at any node in system. Systems must be designed to handle these failures without impacting overall performance or availability of system.
4) Complexity: Distributed computing systems are complex, requiring expertise in multiple areas such as networking, security, and software engineering. This complexity can make it difficult to distributed systems. The inherent complexity of managing communication, coordination, and synchronization across multiple independent nodes makes these systems more difficult to design, debug, deploy, and manage.
5) Interoperability: Distributed computing systems often rely on multiple technologies and protocols, which can create interoperability issues. Ensuring that different components of system can communicate with each other is critical for overall performance and reliability of system.
6) Heterogeneity: The Internet enables users to access services and run applications over a heterogeneous collection of computers and networks. Heterogeneity applies to all of the following:
   - networks;
   - computer hardware;
   - operating systems;
   - programming languages;
   - Implementations by different developers.
7) Networking problems: several problems are created by the network infrastructure, which have to be dealt with: loss of messages, overloading etc.
8) Data consistency**:** Ensuring that all nodes have a consistent and up-to-date view of the data is a major challenge, especially during network failures.
9) Network dependency**:** Performance and reliability are tied to the network. Issues like latency, bandwidth limitations, and network partitions can affect functionality.
10) Technical issues: If not planned properly, a distributed system can decrease the overall reliability of computations if the unavailability of a node can cause disruption of the other nodes.

11) **Coordination:** Nodes lack a universal clock, so tasks like ensuring the order of events and managing shared resources require complex synchronization and consensus algorithms
12) Troubleshooting and diagnosing problems in a distributed system can also become more difficult, because the analysis may require connecting to remote nodes or inspecting communication between nodes.

1.8. Key benefits

1) **Improved performance:** Workloads can be broken down and processed in parallel across multiple nodes, leading to faster completion times for computationally intensive tasks.
2) **High availability:** By eliminating a single point of failure, distributed systems ensure that services remain operational even if some components fail.
3) **Cost-effectiveness:** These systems can use clusters of less expensive commodity hardware instead of a single, powerful machine.
4) **Geographic distribution:** Deploying nodes in different locations allows services to be placed closer to users, reducing latency.

1.8. Trends influencing Distributed Systems & Future Trends

Distributed systems are undergoing a period of significant change and this can be traced back to a number of influential trends:

1) the emergence of pervasive networking technology;
2) the emergence of ubiquitous computing coupled with the desire to support user mobility in distributed systems;
3) the increasing demand for multimedia services;
4) the view of distributed systems as a utility.

1.8.1. Future Trends in Distributed Computing Systems

There are several emerging trends in distributed computing systems that are likely to shape future of technology. These trends include

1) Serverless Computing: Serverless computing is a model where cloud provider manages infrastructure and automatically scales resources based on workload. Developers can write code that runs as functions without need to manage servers, operating systems, or runtime environments. Serverless computing is ideal for applications with unpredictable workloads.
2) Containerization: Containerization is a lightweight virtualization technology that allows developers to package applications and their dependencies into a single unit. Containers can run on any operating system or cloud provider, making it easier to move applications between environments. Containerization improves application portability, scalability, and reliability.
3) Blockchain: Blockchain is a distributed ledger technology that enables secure, transparent, and tamper-proof transactions. Blockchain has potential to disrupt industries such as finance, healthcare, and supply chain management. By using a distributed computing system, blockchain eliminates need for a central authority, making it more resilient to cyber attacks and fraud.
4) Edge Computing: Edge computing is a model where data processing is done closer to source, rather than sending it to a centralized cloud. Edge computing reduces amount of data that needs to be sent to cloud, improving efficiency and reducing costs.

5) Quantum Computing: Quantum computing is an emerging technology that uses quantum mechanics to perform computations that are beyond reach of classical computers. Quantum computing has potential to revolutionize industries such as finance, cryptography, and drug discovery. Quantum computing requires a distributed computing system to operate, making it ideal for cloud providers.

1.9. Distributed Systems Fallacies

A fallacy is a flaw in reasoning that makes an argument invalid, misleading, or deceptive, even if it sounds plausible, representing an error in logic or a false belief often used to persuade. Everyone, when they first build a distributed system, makes the following eight assumptions. These are so well-known in this field that they are commonly referred to as the "8 Fallacies".

1) The network is reliable.
2) Latency is zero.
3) Bandwidth is infinite.
4) The network is secure.
5) Topology doesn't change.
6) There is one administrator.
7) Transport cost is zero.
8) The network is homogeneous.

1.10 Design Issues of Distributed Systems

1) **Communication Issues**
   a) **Message Passing:** Message passing is a communication method in distributed systems where processes exchange information by sending messages over a network. It can be synchronous (blocking) or asynchronous (non-blocking) and is essential for inter-process communication.
   b) **Communication Latency and Bandwidth:** Latency refers to the delay in data transmission between processes, while bandwidth is the maximum data transfer rate. Both impact the performance and efficiency of a distributed system.
   c) **Communication Protocols:** Communication protocols define rules for data exchange between nodes, ensuring reliable and secure transmission. Examples include TCP/IP for reliable communication and UDP for faster, less reliable data transfer.
2) **Process Management**
   a) **Process Coordination**: Process coordination manages the interaction and synchronization of processes across nodes, ensuring they work together without conflicts. Techniques like mutual exclusion and leader election help maintain consistency.
   b) **Process Migration**: Process migration involves moving a running process from one node to another to balance load, improve performance, or maintain fault tolerance, optimizing resources**,** and preventing failures.
   c) **Thread Management**: Thread management controls the execution, scheduling, and termination of threads within processes, ensuring efficient resource use, minimizing latency, and maintaining performance in multi-threaded environments.
3) **Data Management**

a) **Data Storage**: Data storage in a distributed system manages data across multiple nodes, ensuring efficient access, scalability, and redundancy. Techniques like partitioning and distributed databases handle large datasets across various locations.
b) **Data Access**: Data access refers to retrieving and manipulating stored data across the network. Efficient access mechanisms ensure quick retrieval, even in systems with large datasets and multiple users.
c) **Consistency and Replication**: Consistency ensures that all data copies are identical across nodes. Replication improves access speed and fault tolerance, with strategies like eventual or strong consistency balancing performance and accuracy.
d) **Data Integrity**: Data integrity ensures data accuracy and consistency throughout its lifecycle, preventing corruption or loss. Techniques like checksums and version control maintain data integrity during storage, transfer and processing.

4) **Fault Tolerance and Reliability**
a) **Failure Detection**: Failure detection identifies when a node, process or link fails, allowing the system to take corrective actions, such as rerouting tasks or triggering recovery, ensuring system reliability.
b) **Redundancy and Recovery**: Redundancy duplicates critical components across nodes for availability, while recovery mechanisms, like data replication and checkpointing, restore normal operations after failures with minimal downtime.
c) **Consensus and Quorum Systems**: Consensus ensures distributed nodes agree on decisions, even with failures. Quorum systems define the majority of nodes needed to reach consensus, maintaining consistency despite faults or network partitions.

5) **Security**
a) Authentication and Authorization: Authentication verifies the identity of users or systems, while authorization controls access to resources based on that identity, ensuring only authorized entities can interact with sensitive data.
b) Cryptography: Cryptography secures communication and data using encryption, decryption and cryptographic keys, ensuring authorized access and data integrity during transmission or storage.
c) Data Privacy: Data privacy protects personal and sensitive information from unauthorized access and exposure, using measures like encryption, access control and secure storage to keep data confidential.

6) **Scalability and Modularity**
a) **Scalable Architectures**: Scalable architectures allow a distributed system to handle growing workloads by adding resources like servers or storage, maintaining performance as demand increases.
b) **Modular Design**: Modular design breaks a system into smaller, independent components that can be developed, deployed and scaled separately, improving flexibility and reducing complexity.
c) **Elasticity**: Elasticity enables a system to dynamically allocate and deallocate resources based on demand, optimizing resource usage and cost-efficiency.

7) **Synchronization and Coordination**

a) **Clock Synchronization**: Clock synchronization ensures all nodes in a distributed system share a consistent time, crucial for coordinating tasks and time-sensitive operations. Techniques like NTP are commonly used.
b) **Leader Election**: Leader election selects a node to coordinate actions and manage resources, ensuring consistency and preventing conflicts in decentralized systems.
c) **Mutual Exclusion**: Mutual exclusion prevents multiple processes from accessing the same resource simultaneously, using synchronization mechanisms to maintain data consistency and avoid conflicts.

8) **Transparency**
   a) **Access Transparency**: Access transparency hides differences in data representation and storage, allowing users to interact with resources uniformly across systems.
   b) **Location Transparency**: Location transparency makes the physical location of resources invisible, enabling users to access them without knowing where they are stored.
   c) **Replication Transparency**: Replication transparency hides the replication of data across multiple nodes, allowing users to access it as if there were a single copy.
9) **Performance**
   a) **Load Balancing**: Load balancing distributes workloads across multiple nodes to optimize resource use, prevent overload and improve system performance by increasing throughput and reducing latency.
   b) **Caching and Cache Management**: Caching stores frequently accessed data for faster retrieval, while cache management ensures data is up-to-date and minimizes access time.
   c) **Latency and Throughput**: Latency is the delay in data transfer, while throughput measures the data processed per unit of time. Reducing latency and increasing throughput are essential for improving performance.
10) **Algorithmic Challenges**
   a) **Distributed Algorithms**: Distributed algorithms are procedures that enable multiple nodes in a system to cooperate, ensuring coordination, consistency and fault tolerance. Examples include consensus algorithms and leader election protocols.
   b) **Global State Management**: Global state management tracks and maintains a consistent view of the system's state across all nodes, ensuring synchronization and preventing conflicts.
   c) **Distributed Synchronization**: Distributed synchronization coordinates processes across nodes, ensuring correct execution order and managing shared resource access. Techniques include mutual exclusion, clock synchronization and leader election.
11) **Application-Specific Design Challenges**
   1) **Mobile Systems**: Mobile systems involve devices like smartphones communicating over wireless networks, facing challenges like intermittent connectivity, power limits and network variability, requiring efficient data management and location-based services.
   2) **Sensor Networks**: Sensor networks consist of low-power devices that collect environmental data like temperature and humidity, often used in smart cities and industrial monitoring for large-scale, real-time data collection.
   3) **Peer-to-Peer (P2P) Systems**: P2P systems are decentralized networks where nodes share resources directly without a central server, used for file sharing and collaborative tasks, with challenges in trust, security and scalability.

4) **Cloud Computing**: Cloud computing provides on-demand access to computing resources like storage and processing power over the internet, reducing physical infrastructure needs. Key challenges include security, resource allocation and ensuring high availability.

12) **Debugging and Monitoring**
   a) **Debugging Distributed Systems**: Debugging distributed systems involves identifying and fixing errors across multiple nodes, requiring specialized tools due to concurrency and complex interactions.
   b) **Event Monitoring**: Event monitoring tracks and records system events across nodes, helping detect issues early and ensuring stability for debugging and optimization.
   c) **Distributed Tracing**: Distributed tracing tracks requests across system components, providing a visual timeline to identify performance bottlenecks, failures or latency issues.

13) **Real-Time Systems**
   a) **Real-Time Scheduling**: Real-time scheduling assigns tasks within strict timing constraints, ensuring timely execution in mission-critical systems like healthcare and aviation to prevent delays and failures.
   b) **Quality of Service (QoS)**: QoS in distributed systems guarantees performance characteristics like bandwidth, latency and reliability, prioritizing critical tasks and ensuring optimal performance in real-time applications

1.11. Use Cases & Examples of Distributed Computing Systems

Distributed systems are used everywhere, acting as the backbone for scalable, reliable, and high-performance digital services. Real-world applications include;

1) Finance and Commerce: Amazon, eBay, Online Banking, E-Commerce websites.
2) Information Society: Search Engines, Wikipedia, Social Networking, Cloud Computing.
3) Cloud Technologies: AWS, Amazon Web services, Salesforce, Microsoft Azure, SAP, Open stack.
4) Entertainment: Online Gaming, Music, youtube.
5) Healthcare: Online patient records, Health Informatics.
6) Transport and logistics: GPS, Google Maps.
7) Intranets, Internet, WWW, email, Google search
8) Social networks (Twitter),
9) Content delivery networks (Netflix),
10) e-commerce platforms (Amazon),
11) Banking systems,
12) IoT networks,
13) DNS (Domain Name Server) - Distributed databases (Apache Cassandra)
14) Mobile and Pervasive Computing
15) Distributed supercomputers, Grid/Cloud computing
16) Airline reservation systems
17) Peer-to-peer networks
18) Sensor networks
19) Smart Grids

1.12. Key Network Types and Technologies for Distributed Systems

1) **Local Area Network (LAN):** Used for high-speed, low-latency communication within a single building or campus, ensuring fast interaction between nodes.
2) **Wide Area Network (WAN):** Connects distributed components over large geographical distances, connecting multiple LANs together.
3) **The Internet/Internetworks:** Acts as the primary infrastructure for global distributed systems (e.g., AWS, web services), allowing interconnected networks to communicate seamlessly.
4) **Peer-to-Peer (P2P):** Decentralized networks where all nodes (peers) act as both clients and servers, commonly used in blockchain and file sharing.
5) **Wireless and Fiber Networks:** Physical media used to connect components, chosen based on performance, noise, and interference requirements.
6) **Cellular Networks:** Support mobile nodes in a distributed system, relying on distributed base stations.

1.12.1. Key Networking Concepts for Distributed Systems

1) **Network Protocols (OSI Model):** Standardized rules for communication covering everything from physical cable signals to application layer requests.
2) **Latency and Bandwidth:** Key performance factors; latency defines the delay in communication, while bandwidth defines the capacity of data transfer.
3) **Routers and Switches:** Subsystems that route packets and handle link failures to maintain high availability.
4) **Quality of Service (QoS):** Ensures reliability and bandwidth for critical communication, crucial for maintaining consistency.
5) **Network Security:** Firewalls, Virtual Private Networks (VPNs), and traffic filtering are used to protect communication channels in distributed environments.

These networks enable different architectural models such as Client-Server, which relies on a centralized server for services, and P2P, which spreads responsibilities across all participants.